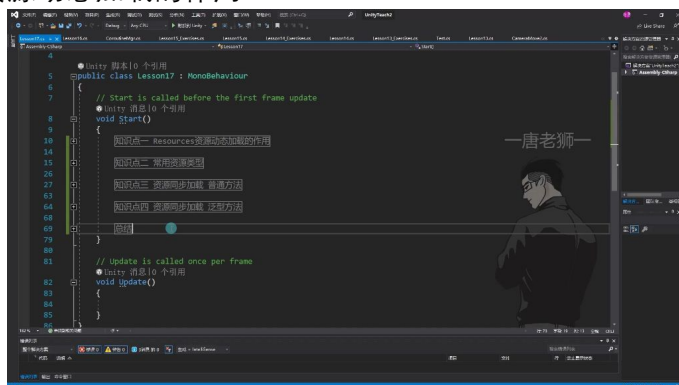


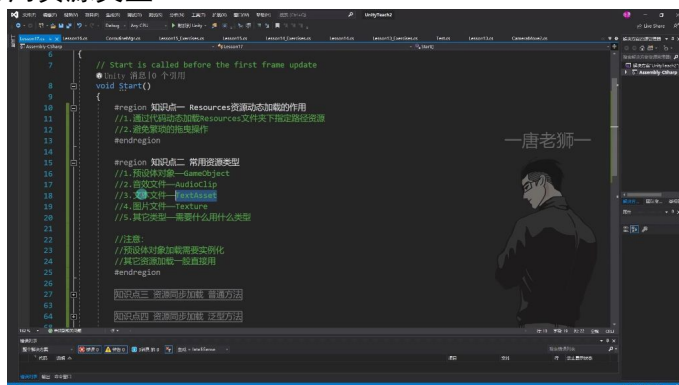
一、Resources资源动态加载 00:02

1. 资源动态加载的作用 00:55

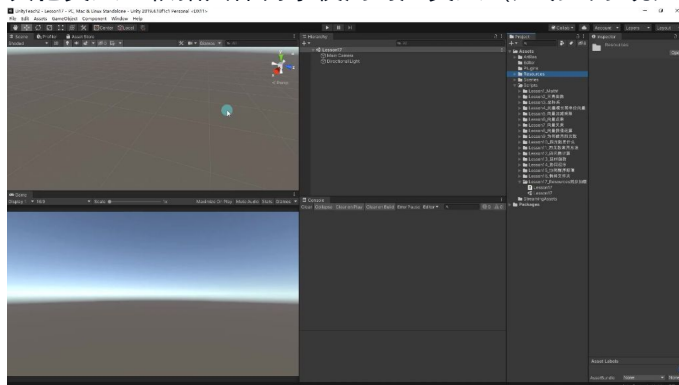


-
- 特殊文件夹：Resources是Unity中的特殊文件夹，名称必须准确无误
- 主要作用：
 - 动态加载：通过代码动态加载Resources文件夹下指定路径资源
 - 简化操作：避免在Inspector窗口手动拖拽关联资源的繁琐操作（特别适用于项目中有大量资源需要关联的情况）

2. 常用资源类型 01:53



-
- 预设体对象：对应GameObject类型，加载后需要实例化才能使用
- 音效文件：对应AudioClip类型（音效切片格式）
- 文本文件：对应TextAsset类型（支持特定格式如XML、JSON等）
- 图片文件：对应Texture类型
- 其他类型：根据具体需求使用对应类型（如动画系统文件等）

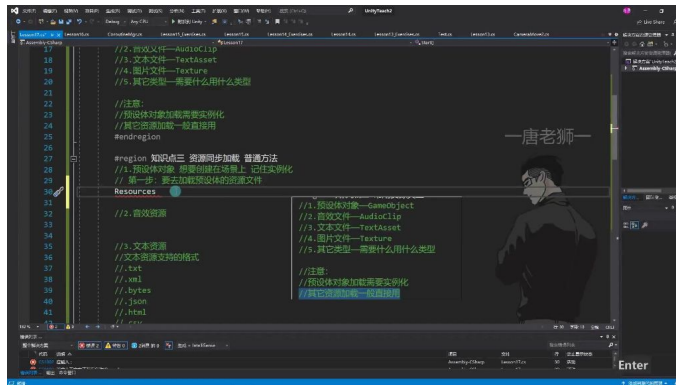


-
- 重要区别：
 - 预设体必须实例化后才能使用

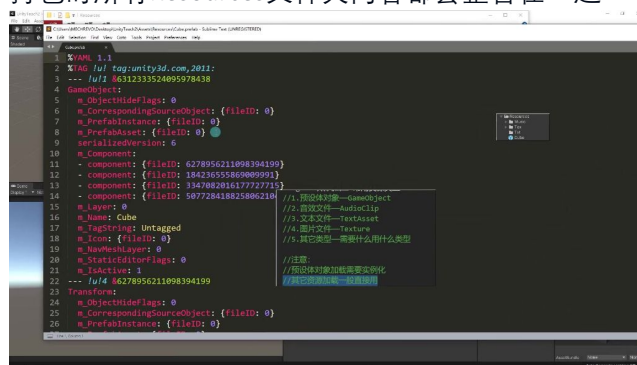
- 其他资源类型一般可直接使用

3. 资源同步加载普通方法 04:08

1) 预设体对象加载 04:37

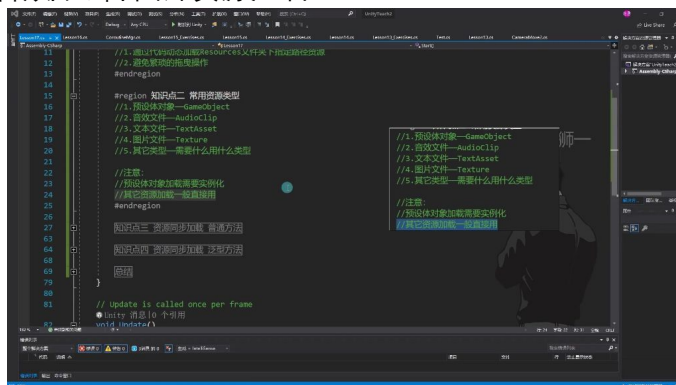


- 加载步骤:
 - 加载资源文件: 使用Resources.Load("Cube")加载预设体资源文件（本质是加载配置数据到内存）
 - 实例化对象: 必须通过Instantiate(obj)将加载的预设体实例化到场景中
- 注意事项:
 - 预设体必须放在Resources文件夹下（工程中可存在多个同名Resources文件夹）
 - 加载后必须实例化才会出现在场景中（实例化对象名称会带有"(Clone)"后缀）
 - 打包时所有Resources文件夹内容都会整合在一起



- 文件本质: 预设体是包含模型关联资源、父子关系等信息的配置文件

2) 音效/文本/图片资源加载 09:07



- 主要资源类型:
 - 预设体对象: GameObject（需实例化）
 - 音效文件: AudioClip
 - 文本文件: TextAsset（支持.txt/.xml/.bytes/.json/.html格式）
 - 图片文件: Texture
 - 其他类型: 按需使用对应类型

- ### 3) 多Resources文件夹处理



- 加载示例:

1) 音效文件准备 11:41



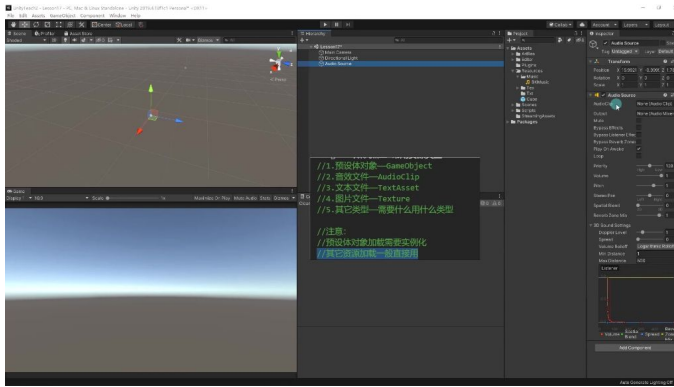
- **文件格式：**支持常见的音频格式如.mp3、.wav等

- ## 2) 音效资源加载步骤 12:12

- **路径规则：**

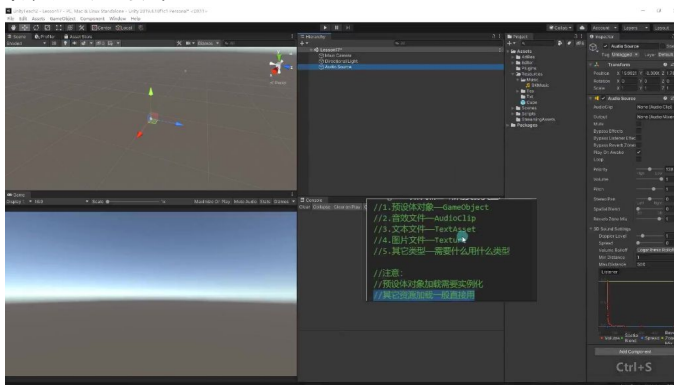
- 两步流程：

- ### 3) 音效资源使用 13:20



-
- 组件要求：场景中需要有AudioSource组件
- 赋值方式：
 - 手动拖拽：将音效文件拖到AudioSource的Clip属性
 - 代码赋值：audioS.clip = obj3 as AudioClip
- 播放控制：通过代码audioS.Play()启动播放
- 重要区别：
 - 预设体资源需要实例化(Instantiate)
 - 音效资源直接赋值即可使用

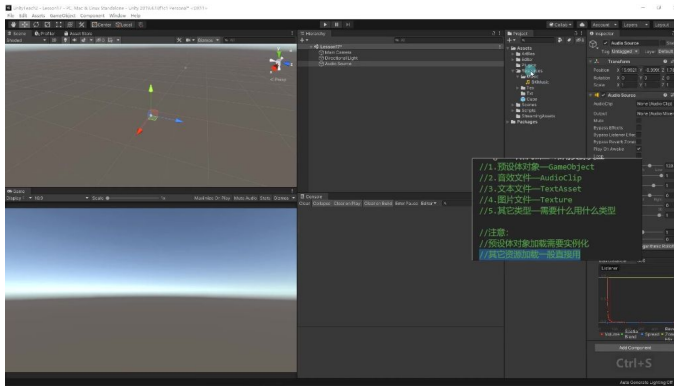
4) 音效播放测试 16:51



-
- 测试步骤：
 - 确保AudioSource组件已关联脚本
 - 运行程序观察Clip属性是否自动填充
 - 检查音效是否自动播放
- 常见问题：
 - 路径错误导致加载失败
 - 忘记赋值导致音效未播放
 - 类型转换失败
- 关键注意：资源加载后必须使用，否则只是占用内存无实际效果

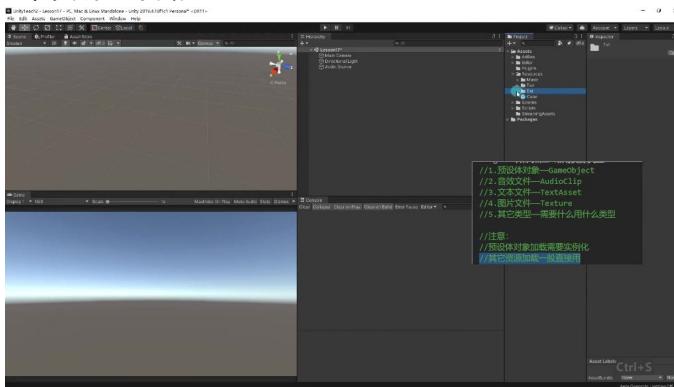
5. 文本资源加载 18:30

1) 文本资源加载方法



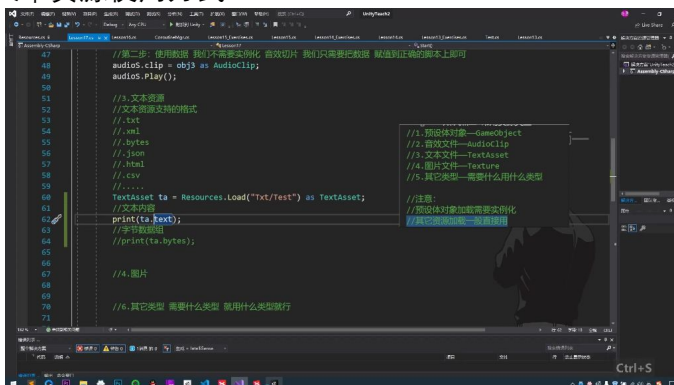
-
- 加载步骤：
 - 使用Resources.Load("路径")加载文本资源
 - 将返回的Object对象转换为TextAsset类型
 - 通过TextAsset的text属性获取文本内容
- 示例代码：

2) 文本资源支持格式

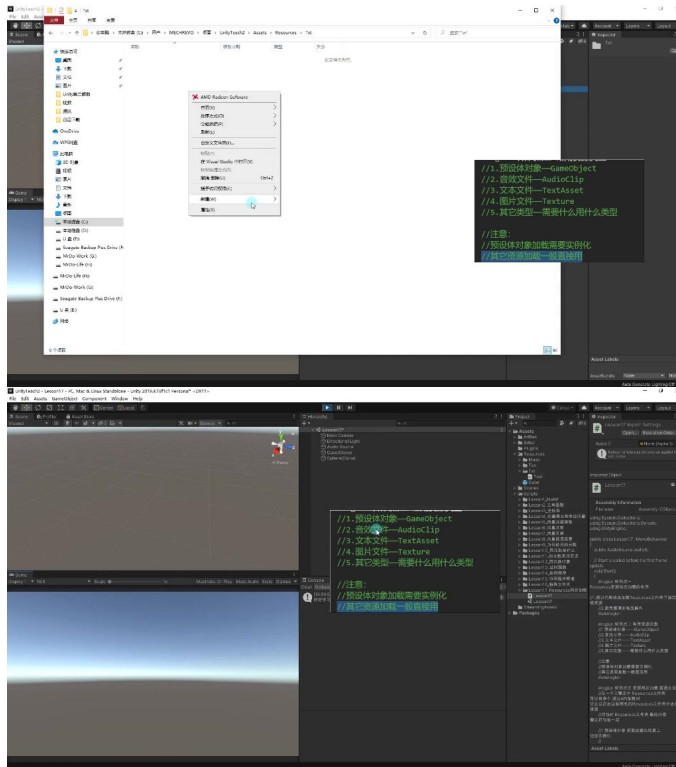


-
- 支持格式：
 - .txt（普通文本文件）
 - .xml
 - .bytes（字节文件）
 - .json
 - .html（使用较少）
 - .csv（使用较少）
- 注意事项：
 - Unity只支持特定后缀的文本文件
 - 最常用的是前四种格式（.txt/.xml/.bytes/.json）
 - 其他格式的文本文件无法被识别和加载

3) 文本资源使用方式



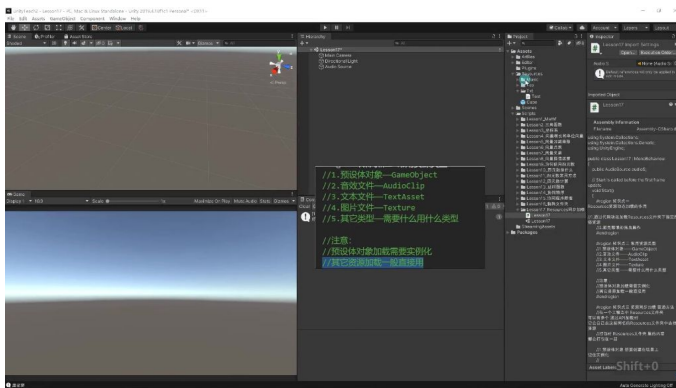
- 内容获取：
 - text属性：获取文本字符串内容
 - bytes属性：获取字节数组（用于二进制数据）
 - 实际应用：
 - 文本内容可直接用于UI显示或逻辑处理
 - 字节数组可用于需要二进制处理的场景
 - 示例：
- 4) 例题：文本资源加载使用 19:53



- 实现步骤：
 - 在Resources文件夹下创建Txt子文件夹
 - 新建test.txt文件并写入内容（如"唐老师好好学习，天天向上"）
 - 使用Resources.Load加载并转换为TextAsset
 - 通过ta.text获取并打印文本内容
- 验证结果：
 - 运行后控制台正确输出文本内容
 - 证明资源加载和使用流程正确
- 注意事项：
 - 路径要正确（区分大小写）
 - 必须使用as进行类型转换
 - 文件后缀必须是Unity支持的格式

6. 图片资源加载 23:15

1) 图片资源加载方法 23:47

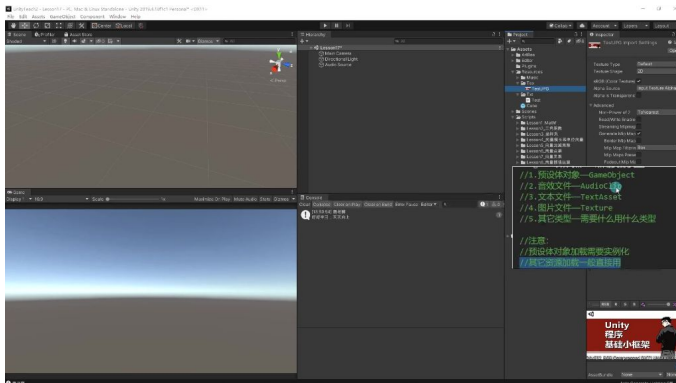


-
- 加载路径：必须准确填写Resources文件夹下的相对路径，路径错误将导致加载失败
- 类型转换：使用Resources.Load加载出来的内容都是Object类型，需要as转换成对应类型（如Texture）
- 示例代码：

```
1 Texture tex = Resources.Load("Tex/TestJPG") as Texture;
```

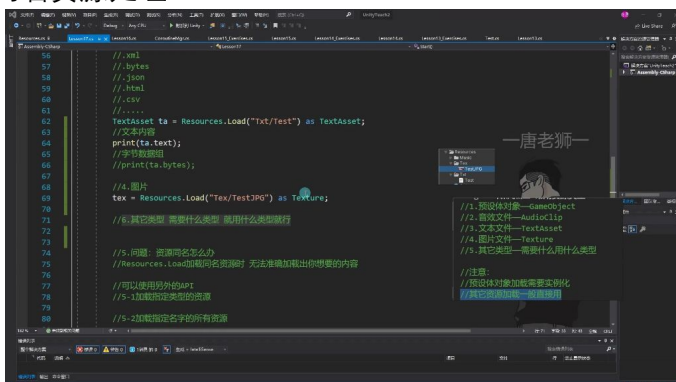
- 显示方法：可通过GUI.DrawTexture方法显示加载的图片资源

2) 资源加载注意事项

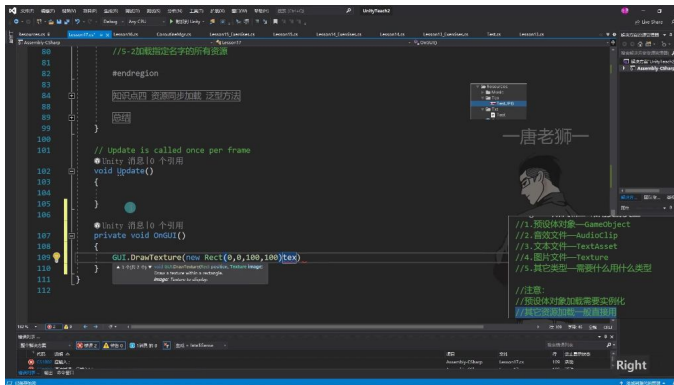


-
- 常见资源类型：
 - 预设体对象(GameObject)
 - 音效文件(AudioClip)
 - 文本文件(TextAsset)
 - 图片文件(Texture)
 - 其他特定类型资源
- 加载区别：
 - 预设体：需要先加载再实例化
 - 其他资源：加载后可直接使用
- 路径规范：路径中不要包含文件扩展名

3) 同名资源处理

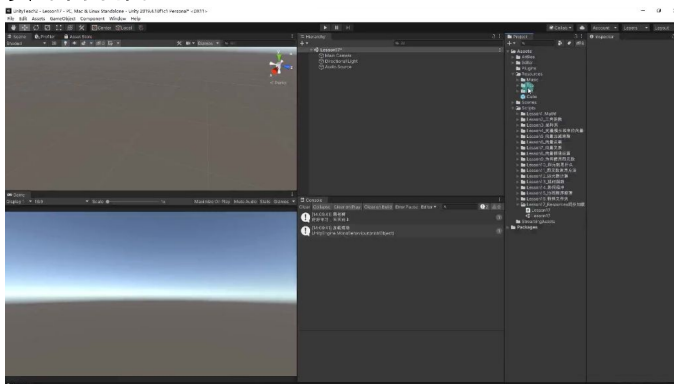


- 问题：Resources.Load加载同名资源时无法准确识别
 - 解决方案：
 - 使用指定类型加载API
 - 加载指定名字的所有资源
 - 推荐做法：避免资源同名，保持资源命名唯一性
- 4) 资源加载流程示例



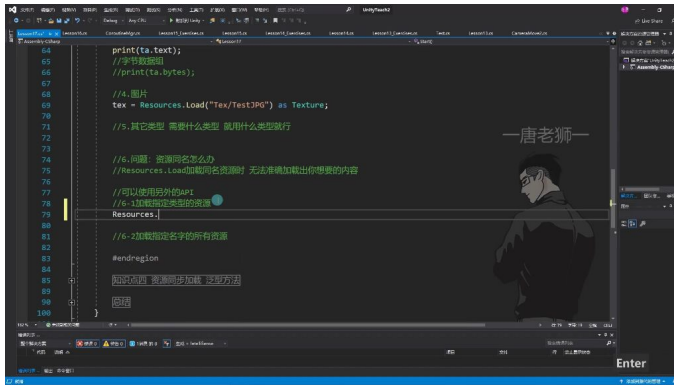
- 完整步骤：
 - 声明Texture类型变量
 - 使用Resources.Load加载图片
 - 类型转换为Texture
 - 在OnGUI中使用GUI.DrawTexture显示
- 关键点：
 - 资源必须放在Resources文件夹下
 - 加载和显示分为两个步骤
 - 需要处理可能的加载失败情况

二、资源同名问题 26:25



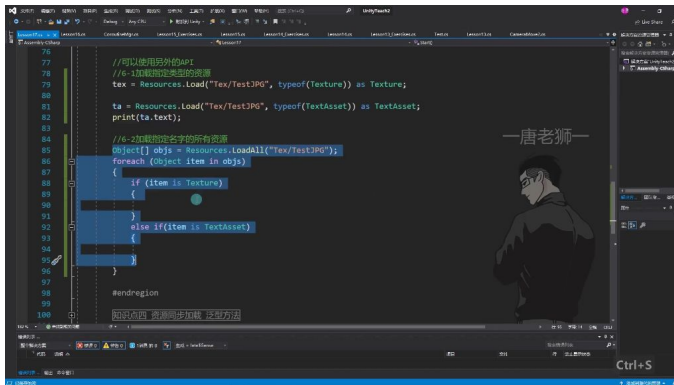
- 问题描述：在Unity中，当同一文件夹下存在同名但类型不同的资源文件时（如TestJPG.png和TestJPG.txt），使用Resources.Load方法加载会出现无法准确识别的问题。
- 产生原因：Resources.Load默认只根据路径加载，无法自动区分同名文件的不同类型。

1. 加载指定类型的资源 27:30



-
- 解决方法: 使用Resources.Load的重载方法, 传入类型参数
 - API格式: Resources.Load(path, typeof(Type))
 - 示例代码:
- 实现原理: 通过System.Type参数明确指定要加载的资源类型
- 注意事项:
 - 返回值仍是Object类型, 需要as转换为目标类型
 - 路径必须正确, 包含Resources文件夹下的相对路径

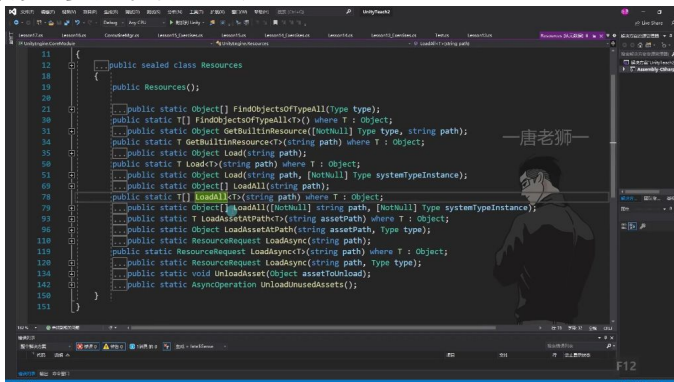
2. 加载指定名字的所有资源 30:13



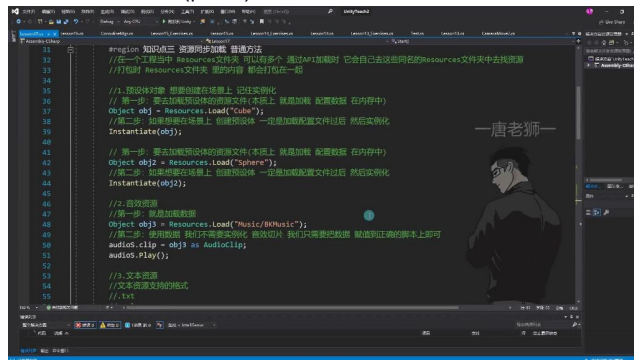
-
- 替代方案: 使用Resources.LoadAll方法
 - API特点: 返回包含所有同名资源的Object数组
 - 示例代码:
- 适用场景: 当需要同时处理同名资源的所有类型时
- 对比选择:
 - Load指定类型: 适合明确知道所需资源类型的情况
 - LoadAll: 适合需要处理多种类型或不确定类型的情况

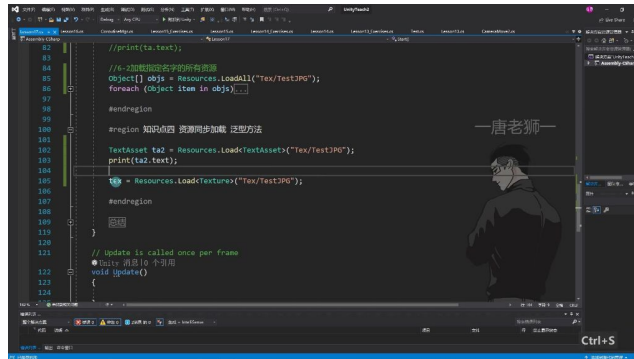
三、资源同步加载 31:24

1. 资源同步加载普通方法 32:13



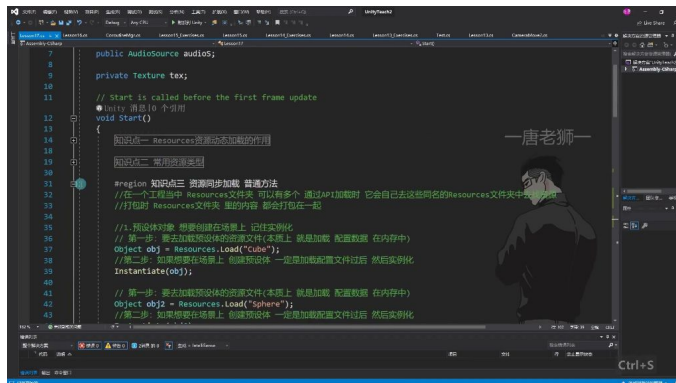
- **核心原则：**资源加载后必须使用，否则会造成内存浪费
- **预设体加载：**
 - 第一步：通过Resources.Load("资源路径")加载配置数据到内存
 - 第二步：使用Instantiate()实例化到场景中
- **音效资源加载：**
 - 直接加载后赋值给AudioSource组件，无需实例化
- **文本资源支持格式：**
 - .txt/.xml/.bytes/.json/.html/.csv等
 - 通过TextAsset类型获取文本内容或字节数组
- **同名资源处理：**
 - 使用Load(path, type)指定类型精确加载
 - 或使用LoadAll(path)加载所有同名资源后遍历筛选



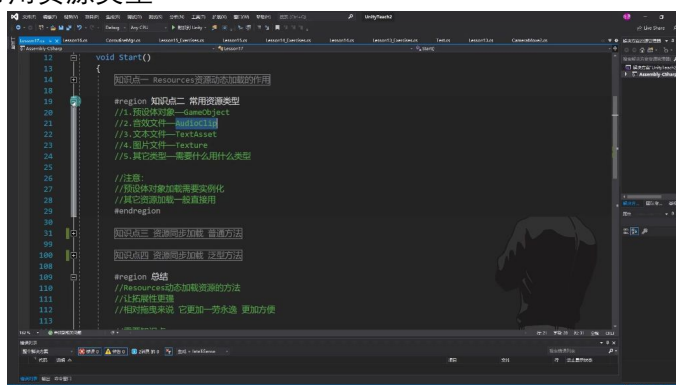


-
- 与传统方法对比：
 - 传统：Texture tex = Resources.Load("路径") as Texture
 - 泛型：Texture tex = Resources.Load<Texture>("路径")
- 核心改进：
 - 泛型方法将类型检查和转换内置到API中
 - 减少代码量，提高可读性和安全性

四、Resources资源动态加载 34:54



-
- 1. 作用与优势
 - 拓展性增强：通过代码动态加载资源，可根据配置文件灵活加载不同资源，避免硬编码
 - 操作便捷性：相比拖拽关联方式更加一劳永逸，减少重复操作
 - 多文件夹支持：工程中可存在多个Resources文件夹，API会自动搜索同名文件夹
- 2. 常用资源类型



-
- 预设体对象：GameObject类型，需实例化后才能使用
- 音效文件：AudioClip类型，需关联到AudioSource组件
- 文本文件：TextAsset类型，可用于配置读取等场景
- 图片文件：Texture类型，需赋值给材质或UI组件
- 其他类型：根据具体需求选择对应资源类型

3. 同步加载方法

1) 普通加载方法

```
1 // 预设体加载示例
2 Object obj = Resources.Load("cube"); // 加载资源配置
3 Instantiate(obj); // 必须实例化才能使用
4
5 // 其他资源加载示例
6 Object obj2 = Resources.Load("Sphere");
```

- **加载本质：**将资源配置数据加载到内存中
- **使用要点：**预设体必须实例化，其他资源需正确赋值使用

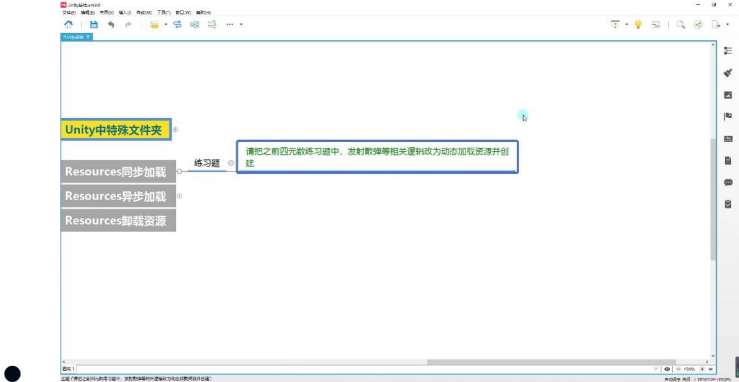
2) 泛型加载方法

- 提供类型安全的加载方式，避免类型转换
- 语法更简洁，推荐使用

4. 注意事项

- **资源必须使用：**加载后必须有实际使用场景，不能只加载不使用
- **预设体特殊处理：**GameObject类型必须实例化才能显示在场景中
- **音效关联：**AudioClip需要关联到AudioSource组件才能播放
- **格式规范：**需严格遵循资源路径和命名规范

5. 练习题



- **改造要求：**将之前四元数练习中的散弹发射逻辑改为动态加载资源方式
- **练习重点：**掌握Resources.Load和Instantiate的配合使用

五、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
Resources资源动态加载的作用	1. 通过代码动态加载Resources文件夹下指定路径的资源 2. 避免繁琐的拖拽操作（如预设体、音效等关联）	易混淆点： Resources文件夹可多级嵌套，且工程中允许多个同名文件夹	★★
常用资源类型	1. 预设体→GameObject 2. 音效→AudioClip 3. 文本→TextAsset（支持TXT/XML/JSON等） 4. 图片→Texture	重点： 预设体需实例化，其他资源直接赋值使用	★★★
资源同步加载普通方法	1. Resources.Load(path)加载任意资源（返回Object）	易错点： 路径需从Resources	★★★ ★

	2. Resources.Load(path, typeof(T))指定类型加载 3. Resources.LoadAll(path)加载同名所有资源	下级开始, 且区分大小写	
资源同步加载泛型方法	Resources.Load<T>(path): 直接返回泛型类型, 无需强制转换 (如 Resources.Load<Texture>("Tex/test.jpg"))	重点: 推荐使用泛型方法, 避免类型转换风险	★★
同名资源处理	1. 通过类型参数区分 (如 typeof(Texture)) 2. 使用LoadAll遍历判断类型	对比维度: 性能 (泛型方法更优) vs 灵活性 (LoadAll可批量处理)	★★★
特殊规则	1. Resources文件夹可存在于工程任意位置 2. 打包时自动整合所有Resources内容	注意: 路径错误会导致加载失败 (返回 null)	★★